

1.1 Task

Relation A)

A)

Superkeys are

EmpID

SSN

Email

EmpID, name

SSN department

Email, phone

B)

Candidate keys

EMPid

SSN

Email

C)

I would choose EMPid as the primary key, because , it is short and simple, it is specifically designed to identify an employee , it doesn't contain sensitive info, easier to use in foreign key relationships , it is stable compared with attributes such as email or phone number .

D)

Yes they can have the same phone number. The sample data shows different phone numbers , but the relation does not state that Phone must be unique. For exmpl , two employees can share a company phone or office phone.

Relation B)

The primary key should be (StudentID, CourseCode, Section, Semester, Year)

Why i chose them

Studentid - Identifies which student is registered

CourseCode- Identifies the course

Section – Identifies the specific section of the course

Semester - allows the same student to take the same course in different semesters

Year- Distinguishies the same semester across different years.

Additional candidate keys task

Based on business rules there are no candidate keys.

Task 1.2

Foreign key relations

Student.AdvisorID to Professor.ProfID

Professor.Department to Department.DeptCode

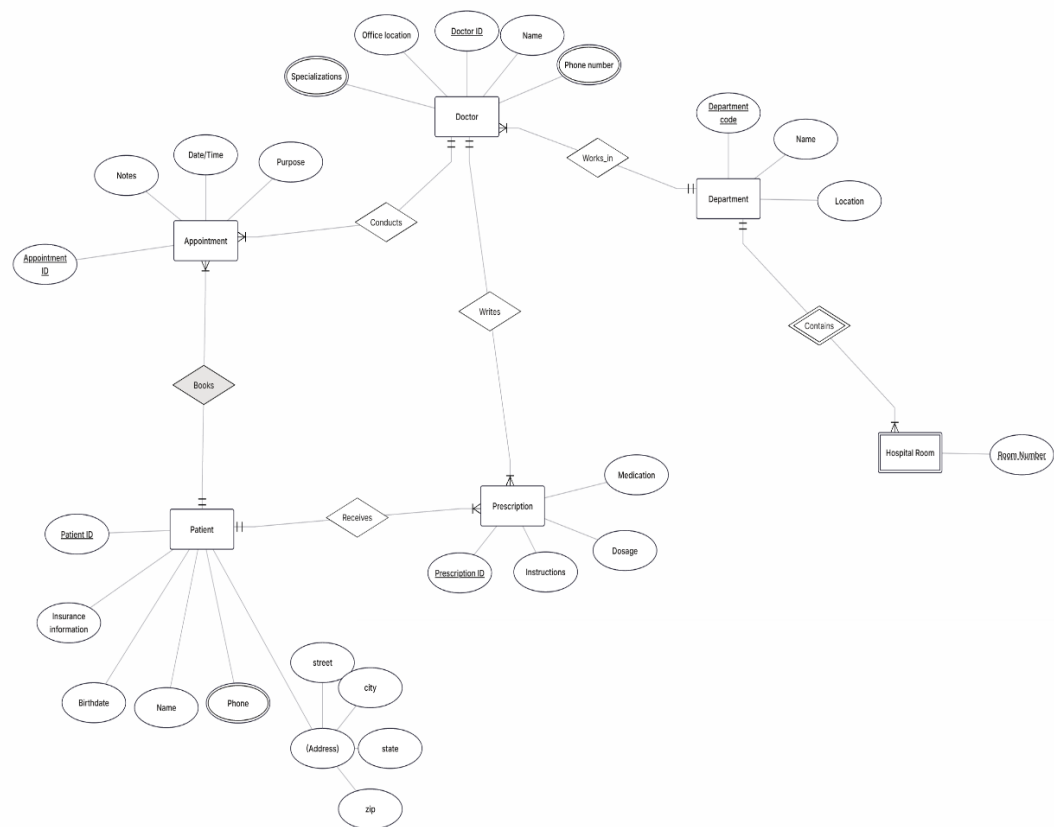
Course.DepartmentCode to Department.DeptCode

Department.ChairID to Professor.ProfID

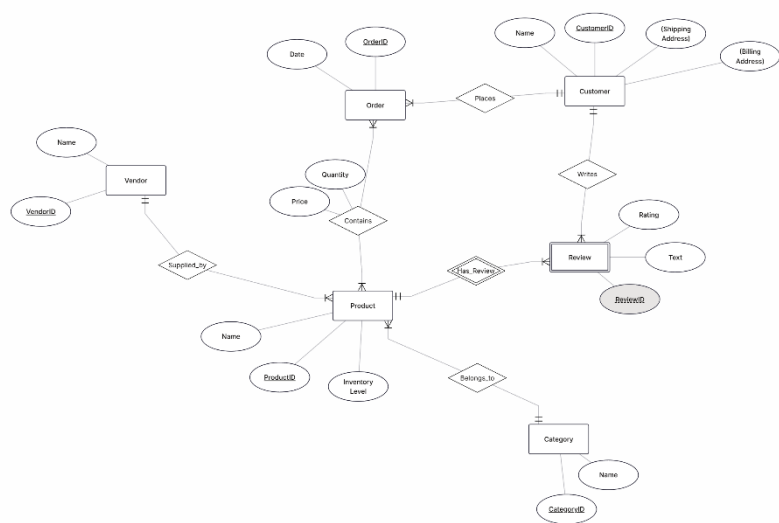
Enrollment.StudentID to Student.StudentID

Enrollment.CourseID to Course.CourseID

Task 2.1



Task 2.2



Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

1. Functional Dependencies

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

(StudentID, ProjectID) $\rightarrow$ Role, HoursWorked

2. Problems

Redundancy: Student and project information is repeated in multiple rows.

Update anomaly: Changing a supervisor's department requires updating multiple rows.

Insert anomaly: A new project cannot easily be added if no student is assigned to it.

Delete anomaly: Deleting the only student assigned to a project may also delete the project's information.

3. 1NF

The table is already in 1NF because all attributes contain atomic values and there are no repeating groups.

4. 2NF

Primary Key: (StudentID, ProjectID)

Partial dependencies:

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

2NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate)

StudentProject(StudentID, ProjectID, Role, HoursWorked)

5. 3NF

Transitive dependency:

ProjectID $\rightarrow$ SupervisorID

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Final 3NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate)

Supervisor(SupervisorID, SupervisorName, SupervisorDept)

StudentProject(StudentID, ProjectID, Role, HoursWorked)

Task 4.2: Advanced Normalization

1. Primary Key

The primary key is:

(StudentID, CourseID, TimeSlot, Room)

It identifies a student's enrollment in a specific course section.

2. Functional Dependencies

StudentID $\rightarrow$ StudentMajor

CourseID $\rightarrow$ CourseName

InstructorID $\rightarrow$ InstructorName

(TimeSlot, Room) $\rightarrow$ Building

(CourseID, TimeSlot, Room) $\rightarrow$ InstructorID

3. BCNF

The table is not in BCNF because several determinants are not superkeys.

For example:

StudentID $\rightarrow$ StudentMajor

StudentID does not uniquely identify a row in the original table.

4. BCNF Decomposition

Student(StudentID, StudentMajor)

Course(CourseID, CourseName)

Instructor(InstructorID, InstructorName)

RoomSchedule(TimeSlot, Room, Building)

CourseSection(CourseID, TimeSlot, Room, InstructorID)

Enrollment(StudentID, CourseID, TimeSlot, Room)

5. Information Loss

The decomposition is lossless. The original information can be reconstructed using joins between the tables. No actual information is lost, but retrieving the complete information requires several joins.

Part 5: Design Challenge

Task 5.1: Real-World Application

1. ER Diagram

Main entities:

Student

Club

Faculty

Membership

OfficerPosition

ClubOfficer

Event

Attendance

Room

Budget

Expense

Main relationships:

Student ↔ Club — M:N

Club → Event — 1:N

Student ↔ Event — M:N

Faculty → Club — 1:N

Club → Room/Event — 1:N

Club → Expense — 1:N

Club → Budget — 1:N

2. Relational Schema

Student(StudentID PK, StudentName, StudentEmail, StudentMajor)

Faculty(FacultyID PK, FacultyName, FacultyEmail, Department)

Club(ClubID PK, ClubName, Description, FoundedDate, AdvisorID FK)

Membership(StudentID PK/FK, ClubID PK/FK, JoinDate, MembershipStatus)

OfficerPosition(PositionID PK, PositionName)

ClubOfficer(StudentID PK/FK, ClubID PK/FK, PositionID PK/FK, StartDate, EndDate)

Room(RoomID PK, RoomNumber, Building, Capacity)

Event(EventID PK, ClubID FK, EventName, EventDate, StartTime, EndTime, Description, RoomID FK)

Attendance(StudentID PK/FK, EventID PK/FK, AttendanceStatus)

Budget(BudgetID PK, ClubID FK, FiscalYear, Amount)

Expense(ExpenseID PK, ClubID FK, ExpenseDate, Amount, Description, Category)

3. Design Decision

I chose to create a separate OfficerPosition table instead of storing positions directly in Membership.

This makes the system more flexible because new positions can be added without changing the database structure.

4. Example Queries

Find all students who are officers in the Computer Science Club.

List all events scheduled for next week with their room reservations.

Show each club's total expenses and remaining budget.